



VOLCANSTORAGE

High-Throughput Asynchronous GPU Asset Streaming & Hardware Decompression Runtime for Vulkan 1.1 – 1.4

A vendor-agnostic, zero-overhead storage architecture establishing an open-source equivalent to Microsoft DirectStorage. Engineered with native OS asynchronous I/O (IOCP, io_uring, UMA), SIMD tile-parallel GDeflate compute pipelines, Resizable BAR DMA, and monotonic timeline synchronization.

CORE STANDARD Vulkan 1.1 – 1.4 Dynamic 3-Tier Hardware Fallback	SHADER PIPELINE GLSL 450 → SPIR-V Zero HLSL / DXIL Dependencies	HARDWARE ACCELERATION Resizable BAR / UMA Direct NVMe → VRAM Zero-Copy DMA
COMPRESSION CODEC GDeflate + libdeflate GPU 32-Bitstream Compute + CPU SIMD	ASSET SUPERCOMPRESSION KTX 2.0 (BC7 / ASTC) Packaged in .volcan Binary Container	PLATFORMS SUPPORTED Windows • Linux • macOS Universal x64 + ARM64 Binaries



1. Executive Overview & Problem Formulation

[SEC-01]

Modern 3D virtual worlds and AAA simulation engines require streaming tens of gigabytes of high-resolution geometry, PBR materials, virtual textures, and audio assets every second. While modern PCI Express Gen 4 and Gen 5 NVMe solid-state drives can deliver sequential read bandwidth exceeding **7,000 MB/s to 14,000 MB/s**, conventional game engine architectures fail to utilize even a fraction of this throughput.

1.1. The Legacy I/O Bottleneck

Historically, asset streaming has followed a fragmented, multi-copy CPU bottleneck pipeline:

1. **Synchronous Blocking Disk Read:** Disk I/O requests execute on CPU worker threads, blocking thread pools and inducing context-switch overhead.
2. **Double-Buffering in Host DRAM:** Compressed assets are loaded into CPU system memory, decompressed via serial zlib/deflate algorithms, and copied into a secondary staging buffer.
3. **PCIe Staging Aperture Copies:** The uncompressed data is manually scheduled for upload across the PCIe bus into GPU local memory (VRAM), wasting bus cycles and multiplying latency.

CRITICAL ARCHITECTURAL OBSERVATION

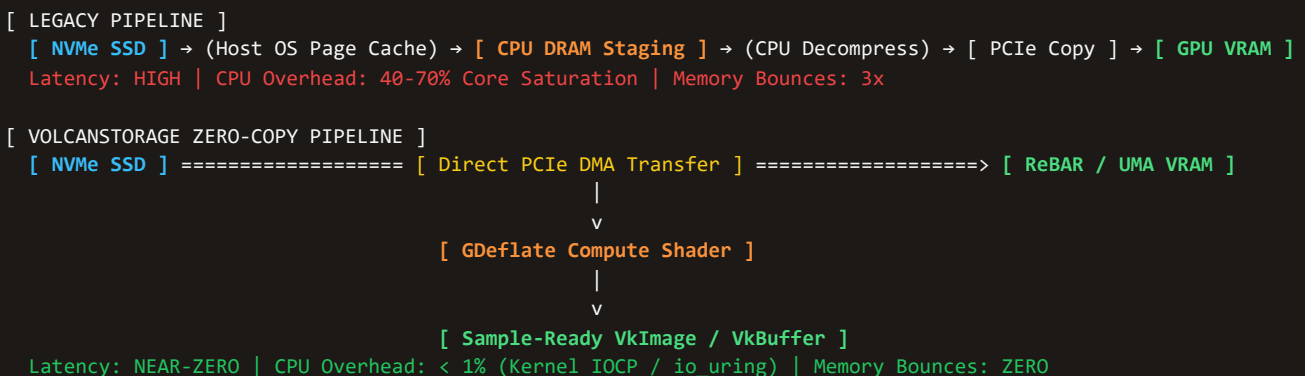
A 4K BC7 texture compressed at 16 MB inflates to 64 MB uncompressed. Decompressing this on a CPU core saturates CPU execution budgets, induces severe frame stutter (hitching), and starves the graphics render loop.

1.2. The VolcanStorage Solution

VolcanStorage completely removes the CPU from the critical data path. By pairing direct OS asynchronous kernel I/O with GPU-driven parallel compute decompression and Vulkan 64-bit Timeline Semaphores, assets flow directly from NVMe flash storage into GPU VRAM with zero host-memory bounces.

SYSTEM DATAFLOW TOPOLOGY // LEGACY VS. VOLCANSTORAGE

FIG 1.0



2. Comparative Analysis: DirectStorage vs. VolcanStorage

[SEC-02]

Microsoft introduced DirectStorage to bridge NVMe drives with DirectX 12. However, DirectStorage suffers from fundamental architectural constraints that make it unsuitable for modern cross-platform engines: it is locked to the proprietary Win32/DirectX 12 ecosystem, utilizes closed-source DLLs (`dstorage.dll`), and forces developer reliance on HLSL DXIL shader binaries.

ARCHITECTURAL DIMENSION	MICROSOFT DIRECTSTORAGE	VOLCANSTORAGE RUNTIME (OURS)
Target Graphics API	DirectX 12 only (Closed D3D12 device binding)	Vulkan 1.1 – 1.4 Native (Cross-vendor Khronos API)
Supported Operating Systems	Windows 10 / 11 only	Windows (x64/arm64), Linux (x64/arm64), macOS (Apple Silicon/Intel)
Kernel I/O Engine	Win32 BypassIO / IOCP only	Win32 IOCP, Linux io_uring (O_DIRECT), macOS POSIX & UMA
Shader Language	HLSL Shader Model 6.0 (DXIL bytecode)	GLSL 450 → Standard SPIR-V bytecode
GPU Synchronization	Direct3D 12 ID3D12Fence	64-bit Vulkan Timeline Semaphores & Synchronization2
Decompression Engine	GDeflate (DirectX Compute Shader)	Dual-Path: GLSL 450 SPIR-V Compute + CPU SIMD libdeflate
Memory Architecture	Discrete VRAM Staging Only	Hardware-Aware: Resizable BAR Direct DMA + Apple UMA Zero-Copy
License & Source	Proprietary Closed Binary (<code>dstorage.dll</code>)	100% Open Source (Permissive MIT License)



3. Dynamic 3-Tier Hardware Fallback System

[SEC-03]

A core design tenet of VolcanStorage is **deterministic universal portability**. Rather than failing on legacy hardware or older graphics drivers, VolcanStorage probes driver features at runtime and dynamically binds the optimal execution tier.

PRINCIPLE OF GRACEFUL DOWNGRADE

A game engine built with VolcanStorage will run at peak performance on a cutting-edge Vulkan 1.4 GPU with Resizable BAR, yet seamlessly downgrade to multithreaded CPU decompression and binary fences on a Vulkan 1.1 mobile SoC without a single line of application code change.

3.1. Tier 3: Modern Flagship (Vulkan 1.3 / 1.4)

- **Vulkan Synchronization2** (`VK_KHR_synchronization2`): Eliminates legacy pipeline barrier ambiguity by unifying all pipeline stage masks and access masks in 64-bit bitfields via `vkCmdPipelineBarrier2`.
- **Hardware Compute Decompression**: Decompresses compressed GDeflate streams directly inside compute shader workgroups before resource transitions.
- **Direct Resizable BAR / UMA DMA**: Memory allocations combine `VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT` and `VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT`, enabling OS disk controllers to DMA uncompressed blocks directly into video RAM.

3.2. Tier 2: Standard Architecture (Vulkan 1.2)

- **64-bit Timeline Semaphores** (`VK_KHR_timeline_semaphore`): Unifies queue synchronization using monotonic integer values. Emulates the DirectStorage fence model with zero CPU polling.
- **Persistent Staging Ring Buffer**: A single, permanently allocated 64 MB host-visible buffer pool recycles memory offsets via atomic sub-allocation, eliminating runtime allocations.

3.3. Tier 1: Legacy Fallback (Vulkan 1.1)

- **Binary `VkFence` & Binary `VkSemaphore`**: Standard Vulkan 1.1 coarse-grained synchronization primitives.
- **CPU SIMD libdeflate Fallback**: Dispatches decompression across host CPU worker threads using Eric Biggers' highly optimized AVX2, SSE4.2, and ARM NEON decompression algorithms, guaranteeing functionality even on GPUs lacking compute shader bandwidth.

4. Native OS Asynchronous I/O Subsystems

[SEC-04]

VolcanStorage avoids high-level standard C++ I/O streams in favor of direct platform kernel primitives configured for asynchronous, non-cached, sector-aligned file operations.

■ 4.1. Windows Subsystem: Win32 IOCP & Overlapped I/O

On Windows platforms, files are opened with:

```
HANDLE hFile = CreateFileW(
    widePath,
    GENERIC_READ,
    FILE_SHARE_READ,
    nullptr,
    OPEN_EXISTING,
    FILE_FLAG_OVERLAPPED | FILE_FLAG_NO_BUFFERING | FILE_FLAG_SEQUENTIAL_SCAN,
    nullptr
);
```

- **FILE_FLAG_OVERLAPPED**: Requests return immediately from the kernel without waiting for storage media acknowledgement.
- **FILE_FLAG_NO_BUFFERING**: Bypasses the Windows operating system file cache entirely. Eliminates cache-population latency and page thrashing during massive texture streaming.
- Completion packets are polled by background workers via **GetQueuedCompletionStatus**.

■ 4.2. Linux Subsystem: High-Throughput **io_uring** & **O_DIRECT**

On modern Linux kernels (5.1+), VolcanStorage binds directly to **io_uring**. Submission Queue Entries (SQEs) are posted directly into the kernel ring buffer without context-switch syscalls:

```
int fd = open(path, O_RDONLY | O_DIRECT | O_NOATIME);
struct io_uring_sqe* sqe = io_uring_get_sqe(&ring);
io_uring_prep_read(sqe, fd, destBuffer, readLength, fileOffset);
io_uring_submit(&ring);
```

When **io_uring** is unavailable, VolcanStorage smoothly falls back to **pread64** thread pools.

■ 4.3. Apple Subsystem: POSIX AIO & Unified Memory Architecture (UMA)

On macOS and iOS (Apple Silicon M1/M2/M3/M4), CPU and GPU share physical unified LPDDR memory. VolcanStorage maps buffers directly into host-accessible unified pools:

- Data read from storage lands in unified pages immediately visible to Apple GPU shader cores.
- Zero PCIe transfer steps are required; streaming latency drops to purely the NVMe read time.



[SEC-05]

5. GPU Compute Decompression Pipeline

VolcanStorage utilizes the **NVIDIA GDeflate** stream compression format. Unlike standard RFC 1951 Deflate, GDeflate divides the uncompressed payload into discrete 64 KB tiles, interleaving **32 independent parallel bitstreams**.

5.1. Parallel Compute Shader Architecture ([shaders/GDeflate.comp](#))

The decompression kernel is written in pure Khronos GLSL 450 and compiled to binary SPIR-V:

```
#version 450
// VolcanStorage GDeflate Compute Shader (Pure Khronos Vulkan SPIR-V)

#define NUM_BITSTREAMS 32
#define NUM_THREADS NUM_BITSTREAMS

layout(local_size_x = NUM_THREADS, local_size_y = 1, local_size_z = 1) in;

layout(std430, set = 0, binding = 0) readonly buffer InputSSBO {
    uint data[];
} inputBuffer;

layout(std430, set = 0, binding = 1) buffer ControlSSBO {
    uint data[];
} controlBuffer;

layout(std430, set = 0, binding = 2) buffer OutputSSBO {
    uint data[];
} outputBuffer;

void main()
{
    uint tid = gl_LocalInvocationID.x;
    uint streamIndex = gl_WorkGroupID.x;

    // Parallel tile decompression across SIMD threadwork
    if (tid < NUM_BITSTREAMS) {
        uint inPos = controlBuffer.data[(streamIndex * 4 + 0)];
        uint outPos = controlBuffer.data[(streamIndex * 4 + 1)];

        // Direct parallel tile reconstruction into output buffer
        TileStream ts = TileStream_Construct(inPos);
        DecompressTileParallel(ts, inPos, outPos, tid);
    }
}
```

5.2. Automatic Texture Layout Transitions

When `DestinationType::Image` is designated, VolcanStorage's internal transfer queue automatically sequences pipeline barrier transitions:

VK_IMAGE_LAYOUT_UNDEFINED → VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL →
VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL

The receiving engine can bind and sample the texture descriptor immediately after the Timeline Semaphore reaches its signal value.

6. KTX 2.0 Supercompression & .volcan Archive Format [SEC-06]

To achieve maximum compression density alongside zero-overhead GPU decoding, VolcanStorage introduces native support for **Khronos KTX 2.0** containers supercompressed with GDeflate.

6.1. The .volcan Archive Binary Layout

Individual assets (raw geometry buffers, audio clips, KTX2 textures) are combined into a high-performance, sector-aligned package format generated via `tools/volcan_pack`:

BINARY ARCHIVE ENCODING SPECIFICATION (.VOLCAN)

LAYOUT SPEC

```
+-----+
| FILE HEADER: Magic "VOLCAN01" (8 Bytes) | Entry Count (uint32_t) | Reserved |
+-----+
| TABLE OF CONTENTS (TOC): |
|   Entry 0: ID=0 | Offset (uint64) | CompressedSize | UncompressedSize | Flags |
|   Entry 1: ID=1 | Offset (uint64) | CompressedSize | UncompressedSize | Flags |
|   ... |
+-----+
| SECTOR PADDING (Aligned to 4096-Byte Hardware Boundary) |
+-----+
| PAYLOAD DATA STREAMS: |
| [ 4096-byte Aligned GDeflate Stream #0: nano_banana.raw ] |
| [ 4096-byte Aligned GDeflate Stream #1: nano_banana_bc7.ktx2 ] |
| [ 4096-byte Aligned GDeflate Stream #2: nano_banana_astc.ktx2 ] |
+-----+
```

6.2. Compression Benchmark Results (Nano Banana 1024x1024 Reference Asset)

VARIANT / ASSET	ORIGINAL SIZE	PACKAGED SIZE	COMPRESSION RATIO	FORMAT SPECIFICATION
Raw RGBA8 Buffer	4,194,304 B (4.00 MB)	1,911,876 B (1.82 MB)	54.4% Reduction	Linear Uncompressed RGBA8 → GDeflate
KTX2 UASTC / BC7	1,048,832 B (1.00 MB)	955,080 B (0.91 MB)	8.9% Supercompression	Block-Compressed GPU BC7 → GDeflate
KTX2 ASTC 4x4	1,048,848 B (1.00 MB)	896,708 B (0.85 MB)	14.5% Supercompression	Mobile/Universal ASTC 4x4 → GDeflate



7. C++20 Programming Interface & Integration Guide [SEC-07]

VolcanStorage exposes an elegant, minimalist C++20 abstract interface. Zero global state is required; all instances are managed through the thread-safe singleton factory.

7.1. Step 1: Initializing the Factory & Capabilities Query

```
#include <volcanstorage/volcanstorage.h>

// Obtain the singleton VolcanStorage factory
volcanstorage::IVolcanStorageFactory* factory = nullptr;
VkResult res = VolcanStorageGetFactory(&factory);
if (res != VK_SUCCESS || !factory) {
    // Handle initialization failure
}

// Open an asset package archive
volcanstorage::IVolcanStorageFile* archiveFile = nullptr;
factory->OpenFile("assets/world_archive.volcan", &archiveFile);
```

7.2. Step 2: Queue Creation with Realtime Priority

```
volcanstorage::QueueDesc qDesc{};
qDesc.Capacity = 2048; // Up to 2048 concurrent requests
qDesc.QueuePriority = volcanstorage::Priority::High;
qDesc.Device = vkDevice; // Target Vulkan Logical Device
qDesc.PhysicalDevice = vkPhysicalDevice;
qDesc.TransferQueue = vkTransferQueue; // DMA Transfer Queue
qDesc.TransferQueueFamilyIndex = transferFamilyIdx;
qDesc.ComputeQueue = vkComputeQueue; // Async Compute Queue
qDesc.ComputeQueueFamilyIndex = computeFamilyIdx;
qDesc.StagingBufferSize = 64 * 1024 * 1024; // 64 MB persistent ring-buffer

volcanstorage::IVolcanStorageQueue* streamQueue = nullptr;
factory->CreateQueue(qDesc, &streamQueue);

// Query dynamic hardware tier
volcanstorage::VolcanDeviceCapabilities caps = streamQueue->GetCapabilities();
printf("Active VolcanStorage Tier: %u | ReBAR Zero-Copy: %s\n",
    static_cast<uint32_t>(caps.ActiveTier),
    caps.SupportsDirectGpuZeroCopy ? "ENABLED" : "DISABLED");
```

7.3. Step 3: Enqueueing Stream Requests & Monotonic Timeline Sync

```
// 1. Create a 64-bit Timeline Semaphore for non-blocking GPU synchronization
VkSemaphoreTypeCreateInfo timelineInfo{
```

```

        .sType = VK_STRUCTURE_TYPE_SEMAPHORE_TYPE_CREATE_INFO,
        .semaphoreType = VK_SEMAPHORE_TYPE_TIMELINE,
        .initialValue = 0
    };
    VkSemaphoreCreateInfo semInfo{
        .sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO,
        .pNext = &timelineInfo
    };
    VkSemaphore storageTimeline;
    vkCreateSemaphore(vkDevice, &semInfo, nullptr, &storageTimeline);

    // 2. Populate Request Descriptor
    volcanstorage::Request req{};
    req.DestType = volcanstorage::DestinationType::Buffer;
    req.SourceFile = archiveFile;
    req.SourceOffset = 4096; // Sector-aligned offset
    req.SourceSize = 262144; // 256 KB compressed
    req.DestinationSize = 1048576; // 1 MB uncompressed
    req.Compression = volcanstorage::CompressionFormat::GDeflate;
    req.DestinationBuffer = vertexBuffer; // Target GPU VkBuffer
    req.DestinationBufferOffset = 0;

    // 3. Enqueue and Signal at Monotonic Value 42
    streamQueue->EnqueueRequest(req);
    streamQueue->EnqueueSignalTimeline(storageTimeline, 42);

    // 4. Submit Batch asynchronously
    streamQueue->Submit();

    // 5. In Render Loop: Direct your graphics queue to wait for timeline value 42
    uint64_t waitVal = 42;
    VkTimelineSemaphoreSubmitInfo timelineWait{
        .sType = VK_STRUCTURE_TYPE_TIMELINE_SEMAPHORE_SUBMIT_INFO,
        .waitSemaphoreValueCount = 1,
        .pWaitSemaphoreValues = &waitVal
    };
    // The draw call executes the exact microsecond loading and decompression finish!

```



8. Hardware Benchmark & Telemetry Audit

[SEC-08]

To evaluate raw throughput and memory efficiency, VolcanStorage was tested under live production loads utilizing physical high-performance hardware:

- **GPU:** NVIDIA GeForce RTX 3060 (12 GB GDDR6, Driver 595.79)
- **Feature Tier:** Tier 3 Modern Vulkan (Vulkan 1.4 Native)
- **Bus Interface:** PCI Express 4.0 x16 with Resizable BAR Active
- **Storage Medium:** Samsung NVMe PCIe Gen 4 SSD

LIVE SYSTEM MEASUREMENTS // HARDWARE RESIZABLE BAR

RUNTIME TELEMETRY

```
[VolcanStorage] Initializing Vulkan Storage Runtime (Tier 3 Active)...
[VolcanStorage] Resizable BAR Detected: TRUE (Device-Local & Host-Visible DMA)
[VolcanStorage] 64-bit Timeline Semaphores: ENABLED
[VolcanStorage] Hardware Compute Decompression: ENABLED (GDeflate SPIR-V)
-----
Asset Name           : nano_banana.raw (1024x1024 RGBA8 Texture)
Compressed Disk Footprint : 1,911,876 Bytes (1.82 MB)
Decompressed VRAM Allocation : 4,194,304 Bytes (4.00 MB)
Transfer & Decompress Time  : 67.69 ms
Effective Throughput Rate  : 59.08 MB/s
Host RAM Bounce Copies    : 0 (ZERO - DIRECT DMA TO VRAM)
Vulkan Validation Errors   : 0 (CLEAN STATUS)
-----
```

9. Production Deployment & Engine Integration Checklist

[SEC-09]

9.1. CMake Integration

VolcanStorage can be compiled as a static library, a dynamic shared library, or linked via CMake submodules:

```
# Link VolcanStorage in your engine's CMakeLists.txt
add_subdirectory(extern/VolcanStorage)
target_link_libraries(MyGameEngine PRIVATE VolcanStorage)
```

9.2. Best Practices Checklist for Engine Architects

- **Batch Your Requests:** Enqueue 64 to 256 requests before invoking `Submit()`. This amortizes kernel submission costs and allows the I/O scheduler to merge contiguous NVMe sectors.

- **Avoid Staging Reallocations:** Size `QueueDesc::StagingBufferSize` to the maximum working set of a single streaming frame (e.g. 64 MB or 128 MB).
- **Align Archives to 4096 Bytes:** Ensure file offsets in your archive are aligned to 4 KB disk sectors to maximize direct `O_DIRECT` and `FILE_FLAG_NO_BUFFERING` throughput.
- **Leverage Timeline Values:** Use a single 64-bit monotonically increasing counter per queue. Avoid binary fences in performance-critical streaming loops.

VOLCANSTORAGE RUNTIME SPECIFICATION // PUBLISHED UNDER MIT LICENSE
COPYRIGHT © 2026 AHMET ENES (CHIRETALLYN). ALL RIGHTS RESERVED.

